

Constant-Time Ancestry Under Mutation

A fully-dynamic tree labeling, benchmarked against static $O(1)$ labelings, ORDPATH, and classical dynamic-tree structures on isolated and interleaved read/write workloads.

INTERLEAVED · $n = 1M$ · VS ORDPATH

1.4x

novel **48-133 ms** vs ORDPATH **77-180 ms** · 100k-op streams, 1-20% writes

P99 · 20% WRITES

500 ns

bounded read tails under mutation; survivors trail at μ s-ms

STATIC · $n = 1M$

DNF

dfs_order and lazy schemes DNF the 60s cap (rebuild cliff)

* interleaved: random tree, $n = 1M$, fixed-seed mixed stream · isolated ops: $n = 10,000$, Criterion fast profile

Read-heavy hierarchies (file systems, compiler IRs, dependency graphs) query ancestry far more often than they mutate structure. Static labelings answer those queries in $O(1)$ — until a mutation forces an $O(n)$ rebuild. This paper demonstrates the point in between: $O(1)$ queries that survive mutation incrementally.

01 CORE CAPABILITY

$O(1)$ Ancestry Resolution. Constant time without traversal, independent of depth and shape, within a fixed size ceiling (a known label family). The dynamic variants carry the novelty claim; cost not yet characterised.

02 MUTATION

Local, not Global. Subtree moves are $O(M)$ in the moved subtree, not $O(N)$ globally. Cache-friendly SoA flat buffers and zero allocations keep the constant factor near memory-bandwidth bound.

03 WORKLOAD

Read-heavy hierarchies. Many ancestry, containment, and subtree queries per mutation; latency budget in the microsecond range.

DRAFT STATUS AND LEGAL DISCLAIMER

Status. This document is an early-stage draft (Preprint, April 2026) released publicly for the purpose of peer review, community feedback, and technical discussion. The methodologies, theoretical complexities, and empirical benchmarks presented herein are subject to revision.

Liability. The algorithms, concepts, and any associated reference implementations are provided “as is”, without warranty of any kind, express or implied, including but not limited to the warranties of merchantability, fitness for a particular purpose, and noninfringement. In no event shall the author be liable for any claim, damages, or other liability, whether in an action of contract, tort, or otherwise, arising from, out of, or in connection with the use, implementation, or scaling of the systems described in this paper.

Copyright. © 2026 Jaun van Heerden. All rights reserved.

ANCESTOR QUERY: RANDOM TREE

The headline test. `is_ancestor(x, y)` asks whether node `x` lies on the path from `y` to the root, the most common containment query in tree-shaped workloads (file paths, scope chains, dominator trees). Random topology approximates organically grown hierarchies where parent assignments are not uniform; $n = 10,000$ keeps the entire tree resident in L3 cache (8 MB on the test machine) so the measurement isolates algorithmic cost from memory pressure. Trees pre-built; only the 5,000-query batch is timed.

RANDOM TREE · $N = 10,000$ · 5,000 QUERIES · LOG SCALE

Novel 113 μ s vs lazy_dfs 270 μ s (2.4 $\times$). Textbook LCT at 90 ms is context, not competition.



variant_a leads at 113 μ s (44 million single-core ancestry checks per second, 23 ns per query). The operative comparison is **lazy_dfs** at 270 μ s — a 2.4 $\times$ gap against the closest static $O(1)$ -query rival. **link_cut** at 90.1 ms (795 $\times$) is reported for continuity with the dynamic-tree literature, but it is the wrong baseline for this operation: an LCT solves fully-dynamic path aggregation on arbitrary trees, and `is_ancestor` is not even a primitive of its interface — it must be synthesized from `access/``findroot`, paying the usual amortised $O(\log n)$ with splay constants. Beating textbook LCT by 795 $\times$ establishes constant-factor headroom, not a result.

Lazy structures (**lazy_dfs**, **lazy_euler**) place high in this view because their query times exclude the $O(n)$ index rebuild paid at construction; the build is not timed in this batch. Any mutation invalidates the index and the build cost reappears at that point. **dfs_order** — interval labeling with in/out DFS timestamps, two integer compares per query, the textbook static answer to exactly this problem — is implemented in the suite but absent from this chart. A revision must include it in every headline view. **(Benchmark needed.)**

METHODOLOGY

Harness

Criterion 0.5 (Rust). Configurable warmup then independent timed samples. Bootstrap-estimated statistics. **Fast profile**: 10 samples, 200 ms warmup, 3 s window. All values are mean point estimates. Confidence intervals are wider than a 50-sample run.

Why fast over precise. The effect is 100–800×; a CI $\pm 15\%$ from 10 samples cannot flip the ordering. A 50-sample run would tighten the CI without changing a single ranking.

Build costs are reported separately: lazy index structures derive query speed from upfront $O(n)$ construction.

Empirical reference: Tarjan & Werneck, **Dynamic Trees in Practice** (ACM JEA 14, 2009).

Topologies

Three shapes, each isolating a distinct stress: depth, skew, and breadth.

Random. Parent chosen uniformly from existing nodes. Fixed Xorshift64 seed. Realistic baseline; depth $\approx \log n$.

Chain. Each node child of the previous. Maximum depth $n - 1$. Worst case for pointer-climbing; isolates path-traversal cost.

Star. All nodes direct children of root. Depth = 1. Best case for pointer-climbing; reveals which structures rely on depth.

Implementations

Novel Algorithm: multiple profiles. Several configuration profiles are benchmarked, each tuned for a different operational target.

Bounded variants operate within a constrained memory footprint. **Dynamic variants** lift those bounds at a modest per-operation overhead.

Classical $O(\log n)$, 3. Link-Cut Trees, Top Trees, Euler Tour Trees. Theoretical optimum for fully dynamic trees across arbitrary operation mixes.

Lazy / index, 3. lazy_dfs, lazy_euler, cmp_ancestor. $O(1)$ query after $O(n)$ build. Any mutation invalidates the index.

Baselines, 6. naive (parent HashMap), flat_vec, hash_label, mpath, dfs_order, btree_chain.

ANCESTOR QUERIES: CHAIN AND STAR

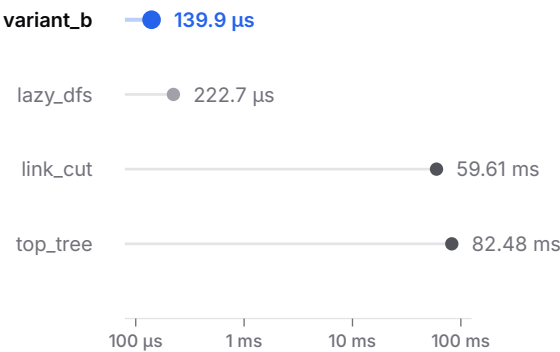
Same operation as the random-tree case, run on two adversarial topologies. Chain forces every ancestor walk to traverse the full tree depth ($n - 1$); star reduces every walk to a single hop. Together they isolate the role of depth in each algorithm’s runtime: a structure whose cost tracks depth swings between these two extremes, while a structure whose cost is decoupled from depth holds steady.

Result. The novel algorithm’s measured time stays within 30% across all three shapes; competitors’ times track tree depth. Against textbook LCT the gaps are 426× (chain) and 126× (star) — context numbers, for the reasons given on the previous page. Against the closest static rival, lazy_dfs on chain runs 223 μs vs the novel 140 μs: a 1.6× gap that this study’s own Limitations classify as suggestive only. The narrowing on star is competitor-side: pointer-climbing structures get faster as depth drops toward 1, while the novel algorithm’s cost is unchanged.

One exception on star: **flat_vec** leads at 71 μs. A flat parent array at depth 1 reduces ancestor lookup to a single array read; this advantage disappears with tree depth. flat_vec on the random tree is 145× slower than variant_a.

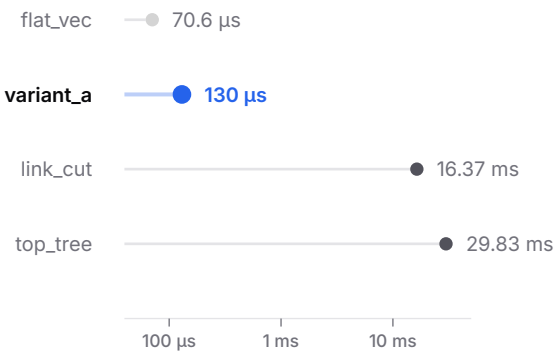
CHAIN · N = 1,000 · LOG SCALE

Novel 140 μs vs lazy_dfs 223 μs (1.6×, suggestive); textbook LCT at 59.6 ms.



STAR · N = 10,000 · LOG SCALE

flat_vec wins at depth 1 (71 μs); novel holds steady at 130 μs.



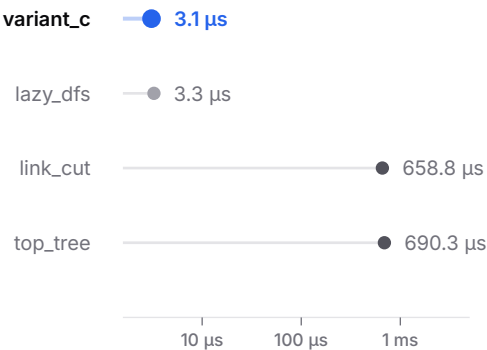
OTHER OPERATIONS

Beyond ancestry, four supporting operations: **depth** (distance from a node to the root), **subtree size** (count of descendants under a node), **build** (construct the full tree from scratch), and **move-subtree** (re-parent a node and all of its descendants). Read operations (depth, subtree size) stress the algorithm’s invariants under repeated lookup; write operations (build, move) stress its mutation cost.

Result. Depth: a statistical tie with lazy_dfs (3.1 μ s vs 3.3 μ s — noise at 10 samples). Subtree size: novel leads, but the gap to lazy_euler (9.9 μ s) is sub-2 $\times$ at Q = 50 — suggestive, not a win, per Limitations. Build and move: simpler structures (flat_vec, lazy_euler) lead at n = 10,000 where cache locality dominates per-operation cost.

DEPTH · RANDOM · Q = 5,000 · LOG SCALE

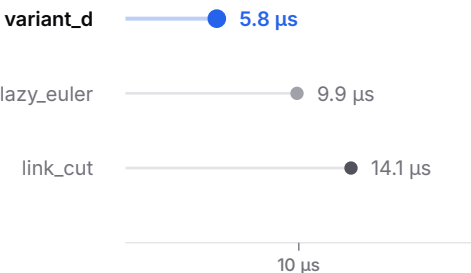
Novel and lazy_dfs at ~3 μ s; classical structures at 660 μ s.



Depth. variant_c (3.1 μ s) and lazy_dfs (3.3 μ s) are a statistical tie at this sample size; lazy_euler follows at 3.7 μ s. link_cut and top_tree: 659–690 μ s. The 45–213 $\times$ margin holds against classical $O(\log n)$ structures only.

SUBTREE SIZE · RANDOM · Q = 50 · LOG SCALE

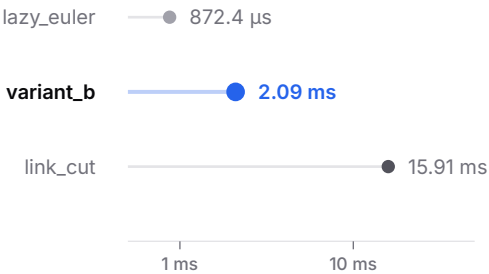
Novel 5.8 μ s vs lazy_euler 9.9 μ s — sub-2 $\times$ gap, suggestive only.



Subtree size. Novel variants occupy the top six results, 5.8–7.3 μ s; lazy_euler follows at 9.9 μ s, link_cut at 14.1 μ s. The leading gap is below 2 $\times$ at Q = 50 with 10 samples — treat as suggestive. Enumeration completes in microseconds across all tested shapes.

BUILD RANDOM TREE · N = 10,000 · LOG SCALE

Novel pays 2.1 ms; lazy_euler leads at 872 μs (mutation-cost regime).



MOVE SUBTREE · N = 5,000 · 50 MOVES · LOG SCALE

flat_vec wins at 96 μs; novel pays 495 μs for invariant maintenance.



Read-side operations extend the novel algorithm’s advantage. Write-side operations are where it pays its costs; structures with no invariant to maintain win on absolute build time. The ranking shifts with the read/write ratio of the workload — which is why the interleaved benchmark in Open Questions, not any chart on this page, is the experiment that decides the thesis.

SUMMARY

Novel best VS fastest other

OPERATION	NOVEL BEST	TIME	FASTEST OTHER	TIME	LCT*	VS LCT*
anc / random	variant_a	113	lazy_dfs	270 μ s	90.1 ms	795×
anc / chain	variant_b	140	lazy_dfs	223 μ s	59.6 ms	426×
anc / star	variant_a	130	flat_vec	71 μ s	16.4 ms	126×
depth / random	variant_c	3.1	lazy_dfs	3.3 μ s	659 μ s	213×
subtree size†	variant_d	5.8	lazy_euler	9.9 μ s	14.1 μ s	2.4×†
build / random	variant_b	2.1	lazy_euler	872 μ s	15.9 ms	7.6×
move subtree†	variant_a	495	flat_vec	96 μ s	807 μ s	1.6×†

* VS LCT compares against a textbook, HashMap-based Link-Cut implementation — context, not the operative baseline: an engineered array-based LCT would run several times faster, and is_ancestor is not even native to the LCT interface. The operative comparison is the FASTEST OTHER column. † Sub-2× and low-sample gaps (subtree size at Q = 50; move subtree, where flat_vec in fact leads) are suggestive only, per Limitations — not wins.

INTERLEAVED WORKLOADS: THE DECIDING EXPERIMENT

Every preceding chart measures operations in isolation — the regime where static labelings look unbeatable because rebuild cost is invisible. The experiment below, one randomized stream of mixed reads and writes, never batched, is the one the thesis rests on.

Harness	Competitors, re-ranked
Single op stream at a fixed read/write ratio, fixed seed, identical stream across structures. Reads: <code>is_ancestor</code> ; writes: <code>move_subtree</code> / <code>add_child</code> (50/50). 100,000 ops per run, 60 s cap (DNF reports ops completed). Per-op latency via monotonic clock (~20–40 ns, uniform). Build excluded, reported separately. Same machine as the isolated-op runs.	dfs_order (the textbook static rival), ORDPATH (closest prior art: prefix labels, no relabel-on-insert), lazy_dfs / lazy_euler , and link_cut as the dynamic floor — textbook, HashMap-based; an engineered LCT would narrow its gap substantially (port pending). Strawmen (naive, hash_label, mpath, btree_chain) dropped.

RANDOM TREE · n = 1,000,000 · 100,000-OP STREAM · TOTAL WALL-CLOCK

MIX R/W	NOVEL (INLINE)	ORDPATH	DFS_ORDER	LAZY_DFS	LINK_CUT
99/1	48.4 ms	78.5 ms · 1.62×	DNF	DNF	3.05 s
95/5	53.7 ms	76.6 ms · 1.43×	DNF	DNF	4.24 s
80/20	133.2 ms	179.8 ms · 1.35×	DNF	DNF	2.79 s

DNF = exceeded the 60 s cap. dfs_order completed 1–12% of each stream — a single write at n = 1M costs it ~0.5–1 s of rebuild, the cliff measured. Lazy schemes DNF on rebuild-on-first-read.

The measured claim. An ORDPATH-class labeling scheme that is depth-robust where ORDPATH degrades catastrophically, with sub-microsecond read tails under concurrent mutation and no rebuild cliff. Three legs, all measured: **(1) no rebuild cliff** — static labelings DNF the moment writes enter the stream; **(2) depth-robustness** — on the 10k chain the novel structure leads ORDPATH 7–34× end-to-end; **(3) bounded tails** — read p99 ≤ 500 ns at n = 1M with up to 20% concurrent writes, while every surviving competitor sits at μs–ms.

INTERLEAVED WORKLOADS: DIAGNOSIS AND HONEST COSTS

The read inversion, found and fixed

The first run of this suite produced a result the isolated-op charts had hidden: at $n = 1\text{M}$, novel **reads lost** to ORDPATH (234 vs 126 ns/query, pure reads) — the in-L3 register-compare advantage did not survive memory pressure, exactly as this paper’s own Limitations warned. Diagnosis ruled out the obvious suspects — label spill (100% of labels at 1M fit one u128; p99 = 71 bits) and footprint (16 MB of labels vs ORDPATH’s 117 MB). The culprit was indirection: three dependent arrays per query endpoint (offset $\rightarrow$ word-count $\rightarrow$ store) against ORDPATH’s two — six potential cache misses vs four. The fix gives every node one fixed 16-byte slot, label length implicit in a leading sentinel bit, spilling past 128 bits. Reads then hit the predicted ceiling — 78.7 ns/query vs 79.2 simulated, ORDPATH at 125.6 — and writes got **faster**: a fixed slot writes in place (the old layout appended and abandoned dead words), and a relabel touches exactly the affected nodes.

What this experiment costs the paper

Build amortisation

Novel build at $n = 1\text{M}$ is 1.8–2.0 s vs ORDPATH’s ~ 1.1 s; repaid from stream speed alone only after $\sim 4\text{--}8$ M ops. Vs `dfs_order` and the lazy schemes payback is effectively instant; vs `link_cut` the novel build is 6–15 $\times$ faster. Short-lived shallow trees favour ORDPATH on build; depth and tails carry shorter horizons.

Deep chains have a memory wall

chain at $n = 1\text{M}$ is skipped **by design**: any prefix-labeling scheme needs $\Theta(n^2)$ total label bits on a 1M-deep chain (≈ 125 GB here, ≈ 4 TB for ORDPATH). The depth claim is depth-robust **within memory budget**, degrading gracefully where ORDPATH degrades catastrophically — not “infinite depth”.

Two profiles, not one

The inline profile owns shallow-at-scale; the prior layout stays 1.7–2.6 $\times$ ahead on the spill-heavy 10k chain (where fast leads ORDPATH 7–34 $\times$, inline 4–19 $\times$). No single variant dominates both regimes yet; unifying them via a flat spill store is open engineering, and until then the profile is chosen by expected depth.

No relabel cascades

Every label rewritten per move is counted, **including** repack cascades: 10–70 labels/move on random (tracks moved-subtree size), 245–478 on the chain (big subtrees, expected widening). No runaway cascades observed.

Baseline fidelity

`link_cut` is textbook, HashMap-based — a loose floor. The ORDPATH here stores plain 8-byte components; canonical ORDPATH bit-compresses labels, so memory gaps vs ORDPATH are implementation artifacts, not results. Compression’s query effect: unmeasured.

Single platform, still

All results come from one laptop-class machine. The indirection-count argument should transfer to server parts — but “should” has already been wrong once in this project; a second-platform run is required.

CANDIDATE APPLICATION: DOMINATOR TREES

Correction. An earlier draft of this page projected “ ≈ 75 minutes per function” from the Link-Cut figure. That number is withdrawn, twice over. First, it priced a cost nobody pays: no production compiler uses LCT-based dominators. LLVM builds dominator trees with semi-NCA (Georgiadis 2005; Georgiadis-Tarjan-Werneck 2006 — a Lengauer-Tarjan 1979 simplification, near-linear in practice) **and**, since 2017, updates them incrementally via DomTreeUpdater (depth-based search; Georgiadis-Italiano-Laura-Santaroni 2016), falling back to full recomputation for large batches. Second, it read the 90.1 ms **5,000-query batch** time as a **per-query** time — a 5,000 $\times$ overstatement. Per query, textbook LCT measures $\approx 18 \mu\text{s}$ and the novel scheme $\approx 23 \text{ ns}$.

Compiler passes (LICM, GVN, SCCP) issue dominance queries — **does block A dominate block B?** — against a dominator tree that must stay valid as passes mutate the CFG. Dominator maintenance is a natural dynamic-trees application that production compilers rejected on constants (Sleator-Tarjan 1983’s own motivating application was network flow — blocking flows in Dinic’s algorithm — not dominators), and an $O(1)$ containment check is a natural fit for the query side. Two things must be true before this scheme applies here, and neither is established:

1 · EXPRESSIBILITY — UNRESOLVED

A CFG edge insertion or deletion can restructure dominance arbitrarily far from the edit site. That is **not a subtree move**, so the $O(k)$ reparent primitive does not obviously support incremental dominator maintenance at all. Whether this labeling can host an incremental dominator algorithm is an open design problem, not a demonstrated capability.

2 · IT MUST BEAT LLVM’S EXISTING PATH — UNMEASURED

The comparison that matters: this scheme’s query + maintenance cost vs LLVM’s semi-NCA **plus its existing incremental updater** (DomTreeUpdater), driven by a **real CFG-edit trace** — not a synthetic subtree-move workload, and not from-scratch recomputation alone. **(Benchmark needed.)**

What this page can stand on. The measured fact is a dominance-style containment check at 23 ns that remains valid across the mutations the structure does support. The hypothesis is that, on functions large enough that semi-NCA rebuilds and DomTreeUpdater batches hurt, maintenance on this structure comes out ahead. Until point 1 is resolved and point 2 is measured, dominator trees are a candidate workload, not an application.

ASYMPTOTIC COMPLEXITY

The table shows the bounded variant, and the bounded variant is the known part: fixed-width packed labels in the bit-packed Dewey / Ahnentafel family, with depth $\times$ bits-per-level held under the label width. $O(1)$ ancestor resolution there does not move the complexity frontier — interval labeling (dfs_order) achieves it statically, and for **growing** trees Dietz 1991 (amortised) and Alstrup-Holm 2000 (worst-case) already pair $O(1)$ queries with $O(1)$ add-leaf. The unclaimed ground is the **move**: growth-only structures do not support re-parenting; pointer-based structures (link-cut, top trees, the Order Indexes of Finis et al., VLDB 2015) support it at $O(\log n)$ per query; and Finis et al. show label-based schemes inherently pay $\Omega(s)$ relabeling per moved subtree of size s — a bound the $O(k)$ move here meets, making it optimal within its class. Lazy caches give $O(1)$ reads only between mutations; any write triggers an $O(n)$ rebuild. Subtree moves are $O(M)$ in the moved subtree; the SoA flat-buffer layout pushes this toward memory-bandwidth bound, which dominates the wall-clock constant factor in practice.

OPERATION	NOVEL	LINK-CUT	LAZY / INDEX	NAIVE
is_ancestor	$O(1)$	$O(\log n)$	$O(1)$ cache	$O(\text{depth})$
depth	$O(1)$	$O(\log n)$	$O(1)$ cache	$O(\text{depth})$
subtree size	$O(1)$	$O(\log n)$	$O(n)$ fresh	$O(n)$
add child	$O(1)$ amort.	$O(\log n)$	$O(n)$ fresh	$O(1)$
move subtree	$O(k)$	$O(k \log n)$	$O(n)$ fresh	$O(k)$
space	$O(n)$	$O(n)$	$O(n)$	$O(n)$

* Novel column shows bounded-variant (fixed-width label) complexity. "add child $O(1)$ amort." amortises over overflow-triggered widenings that relabel descendants. Before the dynamic claim stands: the amortised argument and its worst case (repeated overflow at the widest level); label growth vs the Alstrup-Bille-Rauhe bound (matched by Fraigniaud-Korman 2010); the cost once labels exceed one word, losing the single-register compare. None are characterised here. (Analysis needed.)

WHERE THIS APPLIES

Applicable workloads

Systems storing a hierarchy that query ancestry, containment, or subtree membership more often than they mutate structure:

- File systems** path containment, directory walk queries
- Permission systems** role hierarchies, group membership
- Compiler IR** dominator trees, scope trees, call graphs
- Dependency graphs** transitive dependency checks
- Database engines** operator tree ancestry, query planning

Scope of this study

This benchmark covers ancestry, subtree, depth, build, and move operations. Broader path queries and aggregations fall outside the scope of what was tested.

Lazy index structures achieve $O(1)$ query times but invalidate on any mutation, and static labelings rebuild $O(n)$ per write. The interleaved suite measures exactly this regime: at $n = 1\text{M}$ with 5% writes, `dfs_order` and the lazy schemes exceed a 60 s cap on a 100k-op stream the novel structure completes in 54 ms (see Interleaved Workloads). What was a hypothesis in the first draft of this paper is now a finding, scoped to the measured workloads.

LIMITATIONS

The benchmarks support claims at this scale, on this hardware, under these topologies. Results outside that envelope are extrapolation. The constraints below are the regimes where this study’s findings should not be taken as load-bearing.

Single hardware platform	Synthetic topologies	Sample-size precision
All measurements on an Intel i7-1065G7 (Ice Lake, mobile-class). Server-class CPUs with larger L3 caches and different memory subsystems may shift absolute timings. Relative ordering is expected to hold; not yet verified.	Random, chain, and star bracket the space of tree shapes but do not capture real-world skew patterns (file systems with many shallow paths and few deep ones). Production trees likely fall between random and the extremes.	Fast profile (10 samples) yields wider confidence intervals than a 50-sample run. Effect sizes of 100–800× keep the ordering robust; sub-2× differences should be treated as suggestive only. That bucket includes two results in the Summary: subtree size (1.7× vs lazy_euler, Q = 50) and move subtree (1.6× vs link_cut, where flat_vec in fact leads).
Single-core measurement	Build/move tradeoff	Cache-resident scale (isolated ops)
No multi-threaded contention or scaling tests. Per-core throughput here does not directly translate to multi-core scenarios where lock contention, false sharing, and NUMA effects come into play.	Simpler structures (flat_vec, lazy_euler) lead on absolute build and move-subtree time at n = 10,000. The novel algorithm trades a constant permutation overhead for query-time guarantees; this is an unfavourable tradeoff in write-dominated workloads.	Isolated-op charts use n = 10,000, fully L3-resident. The interleaved suite covers n = 1M — its first run exposed a read regression the in-cache numbers hid, since diagnosed and fixed. Deep chains at 1M are out of reach for any prefix labeling: $\Theta(n^2)$ label memory.

Comparison scope. Baseline implementations follow textbook structures (Sleator & Tarjan 1983; Henzinger & King 1995, with the deterministic counterpart Holm-Lichtenberg-Thorup 2001; Alstrup et al. 2005). The link_cut implementation here is HashMap-based; engineered array-based implementations (KACTL, AtCoder Library) would be several times faster — Tarjan & Werneck (ACM JEA 2009) measured cross-structure gaps of up to 4× and found a naive linear-time implementation beating every $O(\log n)$ structure on low-diameter inputs. Every vs-LCT multiplier in this paper is therefore a loose upper bound on the real gap. The asymptotic argument ($O(1)$ bounded vs $O(\log n)$) does not depend on baseline engineering; only the constant factor does. The “ $O(1)$ ancestor” claim sits adjacent to the dynamic level-ancestor literature — Alstrup-Holm 2000 and Cole-Hariharan 2005 are the nearest theoretical results (growth-only mutation; no moves). Guala-Leucci-Ziccardi 2023, cited in an earlier draft as the nearest competitor, addresses fault-resilience (Faulty-RAM) over leaf-growing trees — a different axis. The interleaved suite now compares directly against dfs_order and ORDPATH, closing the first draft’s largest scope gap. Still missing: an engineered LCT and a level-ancestor structure (Bender-Farach-Colton). **(Benchmarks needed.)**

OPEN QUESTIONS

The first draft of this paper listed six open items, ranked by how much of the claim each carried. Three are now closed by the interleaved suite; what remains is listed second.

Closed since the first draft

✓ Interleaved workloads

Was the experiment the thesis rested on; now run at 99/1, 95/5, 80/20 on $n = 10k$ and $1M$. Static rivals DNF on rebuilds; the novel structure leads ORDPATH $1.35\text{--}1.62\times$ at every $1M$ mix. See Interleaved Workloads.

✓ The right query baselines

`dfs_order` and `ORDPATH` now headline the interleaved comparison — and the first run **lost** reads to `ORDPATH` at $1M$ before the layout fix, which is exactly the information headlining the right rival buys. Level-ancestor structures remain (below).

✓ Scale and memory

$n = 1M$ random measured out-of-cache; the in-cache read advantage did not transfer until the inline layout landed. Footprint measured: 104 MB total at $1M$; 100% of labels fit one `u128` (p99 = 71 bits). Vs-ORDPATH memory claims are withdrawn — see Baseline fidelity.

Still open

1 · Dynamic-variant analysis

The instrumentation shows no widening cascades empirically, but the amortised argument, its adversarial worst case, and label growth vs the Alstrup-Bille-Rauhe lower bounds remain uncharacterised. **(Analysis needed.)**

2 · Remaining baselines

Engineered LCT (KACTL / AtCoder-style — the current `link_cut` column is a loose floor); a level-ancestor structure (Bender-Farach-Colton; Alstrup-Holm 2000); and the later relabel-free XML labelings QED / DDE as related work. **(Benchmarks needed.)**

3 · Unification and platform

One variant for both regimes (a flat spill store should close the deep-chain gap); a second-platform run on server-class hardware; and the multi-parent (DAG) question, which gates any reachability-index application.

Isolated ops: Criterion 0.5 · fast profile · 10 samples · 200 ms warmup · 3 s window
 Interleaved: custom fixed-seed harness · 100,000-op streams · 60 s cap
 Intel Core i7-1065G7 @ 1.30 GHz · 16 GB DDR4 · Rust 1.95.0

JAUN VAN HEERDEN · 2026